Test Project

Cloud Computing

Test Project Designers: Team JP

WSC2026_TP53_SamsungSkills_day2

Contents

Contents **2**
Contents **3**
Task Description **3**
Background **3**
Architecture **3**
Initial state **4**
Three Stages **4**
Stage One: Network Foundation 4
Stage Two: Multi-Cluster ECS 4
Stage Three: Event Pipeline, Normalization 4
Tasks 4
Technical details 4
Service details 5
Reference Document 6

© WorldSkills International

Contents

This Test Project consists of the following documentation/files:

1. WSC2026_TP53_SamsungSkills_day1
2. **WSC2026_TP53_SamsungSkills_day2**
3. WSC2026_TP53_SamsungSkills_day3
4. WSC2026_TP53_SamsungSkills_day4

Task Description

FactoryOps is a SaaS provider serving manufacturing customers. The company is planning to build an internal-only event management platform on AWS. Multiple teams will operate microservices in separate VPCs, but the overall solution must run as one integrated system.

Background

Factory systems generate two categories of events: Order events and Operation events. These events may arrive out of order and may be duplicated, therefore the platform must ensure reliable processing and idempotency.

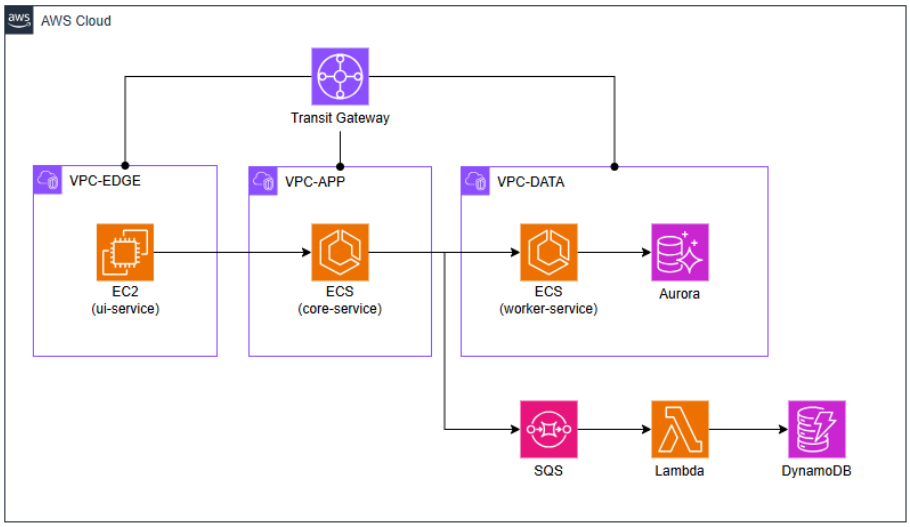
The security policy requires that communication crossing team boundaries (VPC boundaries) must be restricted to the minimum necessary and remain auditable.

Data protection is also critical; ensure data is appropriately protected throughout its lifecycle.

Ensure sufficient operational visibility by collecting appropriate logs and performance indicators for key components.

To support troubleshooting and continuous improvement, implement appropriate observability and request traceability mechanisms.

Architecture



© WorldSkills International

Initial state

An initial (incomplete) architecture concept has been prepared. Your task is to configure AWS services and establish secure connectivity to remove single points of failure where applicable and support reliable event processing.

Three Stages

Stage One: Network Foundation

Implement a secure three‑VPC network foundation. Connect VPCs using Transit Gateway, enforce segmentation, and establish a Cloud Map Private DNS namespace for cross‑VPC service discovery.

Stage Two: Multi-Cluster ECS

Deploy core and worker on ECS. Configure Cloud Map service discovery so that services communicate by FQDN. Provide a single entry endpoint and validate the required communication paths.

Stage Three: Event Pipeline, Normalization

Complete the event processing pipeline for synchronous and asynchronous events. Implement the required validation/normalization logic for the coding component and persist results to the required datastores.

Tasks

1. Carefully review the task description document.
2. Use the us-east-1 region for your architecture setup.
3. Build the network foundation (VPCs, TGW, routing, segmentation).
4. Ensure private workloads can securely access required AWS services while minimizing public internet exposure.
5. Configure Cloud Map Private DNS namespace for service discovery across VPCs.
6. Deploy the provided services to EC2 (ui-service) and ECS (core-service and worker-service).
7. Ensure workload capacity adjusts to demand to maintain performance and availability.
8. Monitor and minimize costs, avoiding excessive scaling or unnecessary resources.
9. Implement event processing for both event categories and ensure idempotent persistence.
10. Implement the read-only API endpoint defined in “Read-only API Specification”.
11. Implement appropriate account-wide security and governance configurations to strengthen the overall security posture and support compliance.
12. Optimize the solution in accordance with the AWS Well-Architected Framework pillars.

Technical details

- Server Application: The provided server application is a pre-compiled binary. Any modifications or use of unofficial binaries will result in disqualification.

© WorldSkills International

- The server application is designed for x86 architecture and optimized for Amazon Linux 2023 and specific Docker containers. All server applications follow these specifications:
  - Listening port: 8080
  - Health check endpoint: `GET /health`
  - Log output: stdout/stderr (must be available for troubleshooting)
  - Environment variables: Set as required for each service
- There are two types of data sent to your system via APIs:
  - Type A data path: core -> worker (validation & save) -> RDS Proxy -> Aurora
    - worker-service validates Type-A events and persists them to Aurora via RDS Proxy
  - Type B data path: core -> SQS -> Lambda (validation & normalization) -> DynamoDB
    - Lambda validates, normalizes Type-B events and persists them to DynamoDB
- Read-only API Specification
  - Method: GET
  - Path: /api/events
  - Response format: application/json
  - The API must return a consolidated view of both Type-A and Type-B datasets.

Service details

VPC

- Create 3 VPCs (VPC-EDGE, VPC-APP, VPC-DATA)
- Create Transit Gateway and attach VPCs
  - Implement segmentation so that direct communication between VPC-EDGE and VPC-DATA is blocked.

EC2

- Create EC2 instance in VPC-Edge
- Deploy ui-service in EC2
- ui-service provides:
  - Web UI for event visualization
  - POST /api/events endpoint: accepts event data and forwards to core-service for processing

ECS

- Create ECS Clusters
  - VPC-APP: app-cluster
  - VPC-DATA: worker-cluster
- Deploy Services
  - core-service in app-cluster
    - Unified API for event ingestion
    - Routes Type-A events to worker-service (synchronous)
    - Routes Type-B events to SQS (asynchronous)
  - worker-service in worker-cluster

© WorldSkills International

- Validates Type-A events
- Persists validated events to Aurora via RDS Proxy

- Register services in Cloud Map so that the following FQDNs resolve:
  - core.svc.internal
  - worker.svc.internal

SQS

- Queue name: events-b-queue
- DLQ name: events-b-dlq

Lambda

- Function name: event-normalizer
- Implement as requirements
  - Validate Type B events
    - orderId must start with ORD-
    - operator must start with op- or operator-
    - operation must be PACK or SHIP
  - Normalize fields (timestamp handling and trimming of text fields).
  - Write to DynamoDB with idempotency.
  - On validation failure or processing error, raise an exception so SQS retry/DLQ behavior can apply.

Aurora

- Aurora PostgreSQL. Do not use serverless.
- Use RDS Proxy, and worker must connect via RDS Proxy
- Required Table

```
CREATE TABLE IF NOT EXISTS events_orders (
  event_id    VARCHAR(64) PRIMARY KEY,
  order_id    VARCHAR(64) NOT NULL,
  amount      NUMERIC(12,2) NOT NULL,
  ts          TIMESTAMP   NOT NULL,
  created_at   TIMESTAMP   NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

DynamoDB

- Create a DynamoDB table with name “events_operations”
- Key Schema:
  - Partition Key: event_id (String)
  - Sort Key: ts (String, ISO8601)

Reference Document

- https://docs.aws.amazon.com/

© WorldSkills International